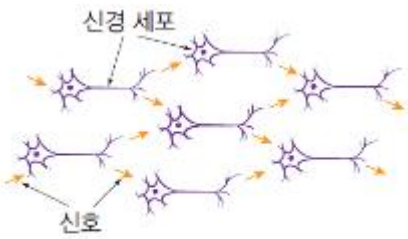
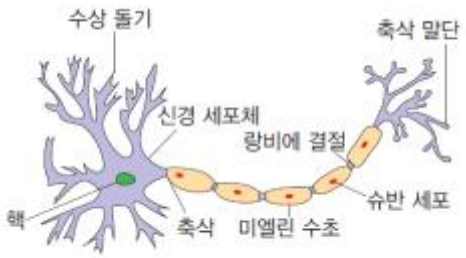
뉴런(Neuron)과 인공신경망(Artificial Neural Network)

인공지능 연구에서 혁신을 이끌고 있는 딥러닝(deep learning)의 시작은 1950년대부터 연구되어 온 인공 신경망(ANN: Artificial Neural Network)이다. 인공 신경망은 생물학적인 신경망에서 영감을 받아서 만들어진 컴퓨팅 구조이다. '스스로 학습하고 생각하는 기계'는 항상 인간의 꿈이었고 1950년대에 사람들은 인간의 두뇌를 본떠서 기계로 만들려고 시도하였다.

인간의 뇌가 정보를 처리하는 방식을 모방하여 인공지능을 만들기 위해 뇌의 기본 단위인 신경 세포에 관심을 가지게 되었다. 동물의 신경 세포인 뉴런은 한쪽 끝에서 다른 쪽 끝으로 전기 신호를 전달한다.



▲ 뉴런의 연결망



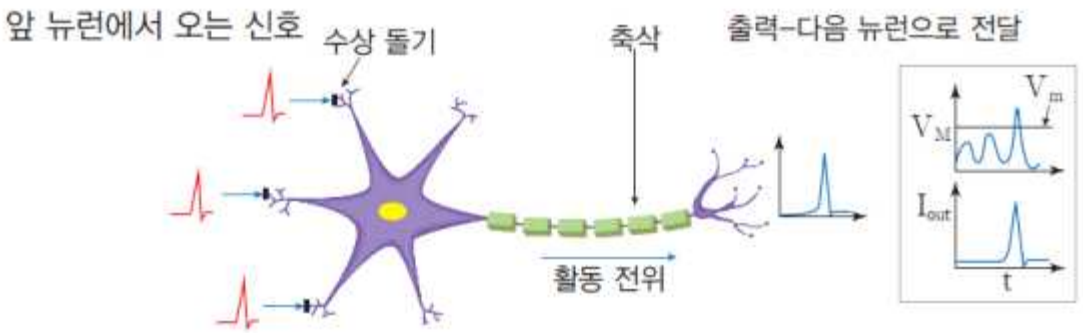
▲ 뉴런의 구조

궁극적으로 동물의 신경 세포인 뉴런은 무엇인가를 인식하고 판단하는데 뉴런의 연합인 신경망을 활용한다. 즉 하나의 뉴런만을 사용하는 것이 아니고 여러 뉴런을 사용하여 서로 연결하여 복잡한 문제를 해결한다.

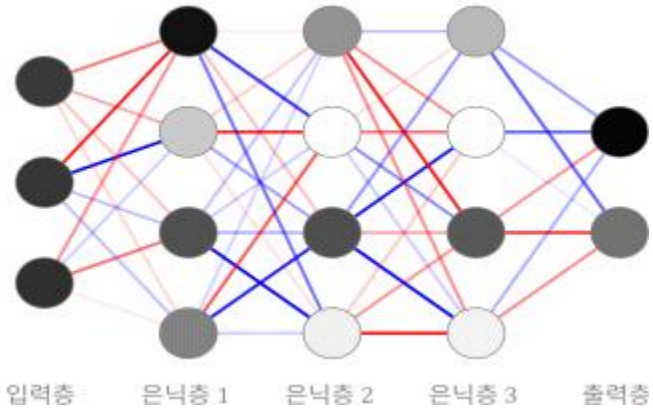
동물 뉴런의 원리에 관해 연구하며 알려진 사실은 다음과 같다.

- 뉴런의 출력값은 단순히 출력 = (가중치×입력값) + 상수, 즉 $y = wx + b$와 같은 선형 관계가 아니다.
- 뉴런은 다른 뉴런으로부터 들어오는 신호의 총합이 어떤 임계값을 넘어서야 출력 신호를 내보낸다.

위와 같은 뉴런의 구조로부터 착안하여, 다양한 함수를 네트워크 형태로 복잡하게 연결하면 어려운 문제에 대한 답을 구할 수 있는 거대한 함수인 '인공 뉴런'을 만들 수 있을 것이라는 기대로 인공 신경망에 대한 연구가 진행되고 있다.



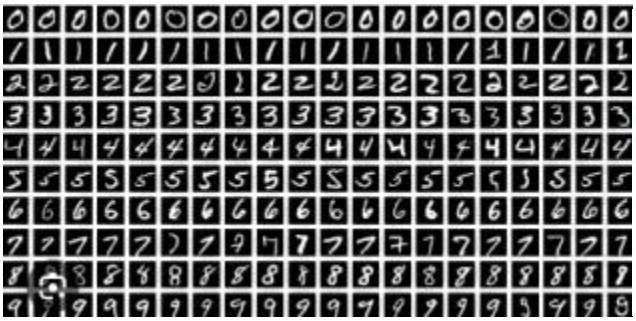
▲ 뉴런에서 신호를 전달하는 방식



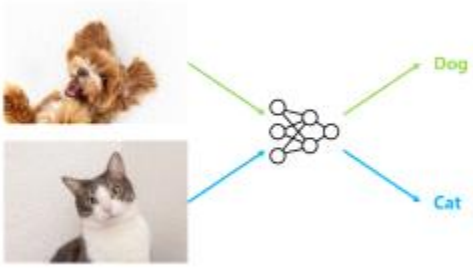
인공 신경망은 유닛(unit), 또는 노드(node)라고 하는 소자들을 모아서 만든다. 이들 소자는 생물학적 뉴런을 느슨하게 모델링 한 것이다. 뉴런은 시냅스(synapse)로 서로 연결되어 있는데 인공 신경망에서도 유닛들은 다른 유닛들과 연결되어 있고, 연결선을 통하여 신호들을 수신한다. 이어서 입력 신호들의 총합에 어떤 함수를 적용하여 출력을 만들어 내고, 이 출력은 다른 유닛들로 전달된다. 위 그림은 인공 신경망을 나타낸 것이다.

하나의 유닛은 아주 단순한 계산만을 할 수 있지만, 수백만 개를 모아 놓으면 아주 복잡한 작업을 할 수 있다. 인공 신경망을 줄여서 간단히 **신경망**이라고 지칭하기로 하자. 신경망의 목표는 단순한 유닛의 조합으로 복잡한 작업을 하게 만드는 것인데 핵심 철학은 **신경망이 데이터로부터 스스로 학습**하게 하는 알고리즘을 만드는 것이다. 즉, 예제로부터 작업을 수행하는 방법을 스스로 배우게 하는 것이다. 우리는 앞서 데이터로부터 선형근사식을 학습하는 회귀 문제를 다루어 보았다. 데이터를 바탕으로 기울기 a와 절편 b를 **경사하강법을 통해 '학습'**하도록 하는 알고리즘 구현이 핵심이었다. 이제 목표는 간단한 회귀 문제뿐만 아니라 여러 특수 목적에 맞는 작업을 할 수 있도록 신경망을 학습시키고 학습된 신경망으로 복잡한 작업을 수행하도록 하는 것이다.

앞서 우리는 여러 가지 종류의 꽃 사진으로 신경망을 학습시켜서 꽃의 종류를 '분류'하는 실습을 해 보았다. 신경망은 손글씨를 인식하는 신경망, 음성인식, 자연어 처리, 소셜네트워크 필터링, 비디오 게임, 의학 진단, 자율주행, 챗봇, 로봇공학 등 수많은 영역에서 활용되며 세상을 바꿀 혁신 기술이 급속도로 발전하고 있다.



손글씨 인식



이미지 인식



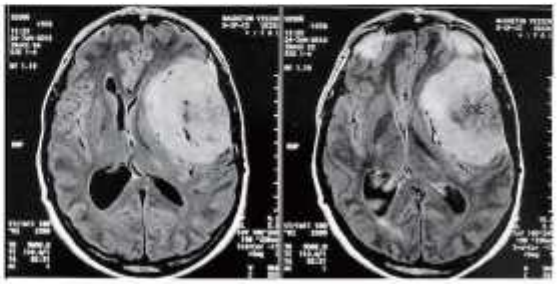
음성 인식



자연어 처리



추천 시스템



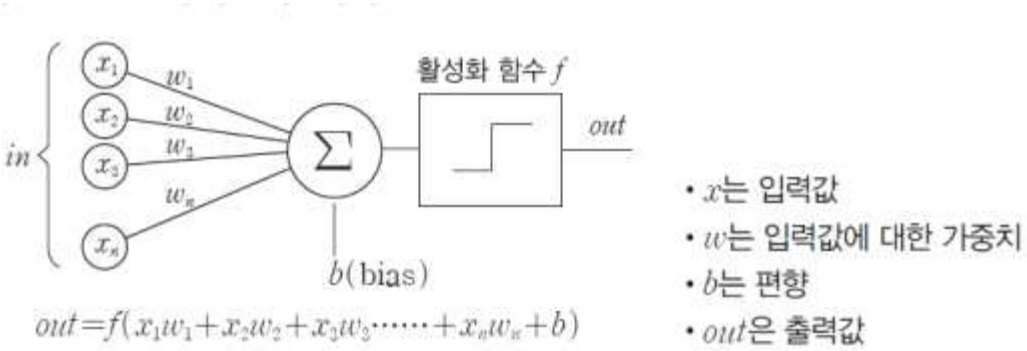
의료 영상 분석

▲ 딥러닝의 다양한 활용 분야

1. 퍼셉트론(perceptron)

신경망의 기본 단위인 퍼셉트론에 대해서 자세히 알아보자. 퍼셉트론은 뉴런의 구조를 모델링하여 프랑크 로젠블라트가 1957년 고안한 알고리즘이다. 퍼셉트론이 발표되었던 1950년대에 신경망은 많은 인기를 끌었으며 스스로 생각하고 말하는 컴퓨터를 만드는 것은 시간 문제로 생각했다. 하지만 그렇게 간단하지 않았다는 것이 이후 밝혀졌고, 여러 난관을 극복해야만 했다. 현대적인 신경망은 많은 수의 유닛(뉴런)을 사용하지만 **퍼셉트론은 하나의 뉴런만을 사용하는 모델**이다. 퍼셉트론은 다수의 신호를 입력으로 받아 하나의 신호를 출력한다. 입력신호 $(x_1, x_2, \cdots, x_n)$에 가중치(weight) $(w_1, w_2, \cdots, w_n)$을 곱하고 편향(bias) b와 합한 값이 임계값을 넘으면 신호를 전달하고, 넘지 않으면 신호를 전달하지 않는 구조를 갖는다.

신호란 전류나 강물처럼 흐름이 있는 것으로 상상하면 좋다. 전류가 전선을 타고 흐르는 전자를 내보내듯, 퍼셉트론 신호도 흐름을 만들고 정보를 앞으로 전달한다. 다만 **퍼셉트론에서 신호는 '흐른다/안 흐른다(1이나 0)'의 두 가지 값을 가질 수 있다.**



▲ 퍼셉트론의 수학적 표현

가중치 $(w_1, \cdots, w_n)$과 편향 b를 구분 없이 모두 가중치라고 부르기도 하며 가중치가 w만을 나타내는지 b까지 포함하는지는 보통 맥락에서 파악할 수 있다. 그림에서는 입력신호를 $(x_1, \cdots, x_n)$, 출력신호를 y라 할 때,

$$y = \begin{cases} 0, & \text{if } w_1x_1 + \cdots + w_nx_n + b \le 0 \\ 1, & \text{if } w_1x_1 + \cdots + w_nx_n + b \le 0 \end{cases}$$

를 나타낸다.

함수 f를

$$f(x) = \begin{cases} 0, & \text{if } x \le 0 \\ 1, & \text{if } x > 0 \end{cases}$$

라 하면

퍼셉트론의 출력은

$$\begin{cases} z = w_1 x_1 + \cdots + w_n x_n + b \\ y = f(z) \end{cases}$$

와 같이 일차함수와 f의 합성으로 나타낼 수 있다.

입력 신호와 가중치를 행렬(또는 행벡터)

$$X = (x_1, x_2, \cdots, x_n)$$
$$W = (w_1, w_2, \cdots, w_n)$$

로 나타내면 행렬곱을 이용하여

$$z = WX^T + b = XW^T + b$$

로 나타내거나, 벡터의 내적을 이용하여

$$z = W \cdot X + b$$

로 나타낼 수 있다. 여기서 X^T는 행렬 X의 행과 열을 바꾼 전치행렬이다. 즉, X는 행벡터를, X^T는 열벡터를 나타낸다.

퍼셉트론은 신호와 가중치의 선형결합을 나타내는 일차함수 $z = W \cdot X + b$와 활성화 함수라고 불리는 어떤 비선형 함수 f의 합성함수 $y = f(W \cdot X + b)$로 생각할 수 있다. W를 가중치, b를 편이(bias)라고 한다. 경우에 따라서는 편이 b 또한 가중치로 생각하기도 하는데 이 경우에는 $W = (b, w_1, w_2, \cdots, w_n)$, $X = (1, x_1, x_2, \cdots, x_n)$ 으로 정의하고 신호의 가중합을

$$z = WX^T = XW^T = W \cdot X$$

로 표시한다.

2. 활성화 함수(activation function)

앞서 설명하였듯이 우리 뇌의 뉴런은 여러 개의 다른 뉴런으로부터의 신호를 전달(입력)받아 합성하여 또 다른 뉴런으로 신호를 전달(출력, fire)하는데, 이때 뉴런은 입력으로 받은 신호의 합성, 가중합 $W \cdot X$ 가 너무 약하면 출력을 내지 않고, 합성된 신호 값이 특정 임계값($-b$)를 넘어야만 다음 뉴런으로 신호를 전달한다. 즉 $z = W \cdot X + b > 0$ 일 때만 그 합성 신호가 죽지 않고 활성화되어 다음 뉴런으로 전달된다. (그렇지 않고 수많은 뉴런 사이의 모든 신호를 전부 살려서 처리한다면 우리 뇌는 터져버릴지도 모르겠다.) 이러한 뉴런에서의 신호의 소멸과 활성 현상을 수리적으로 모델링하기 위해서 여러 종류의 함수들을 사용하

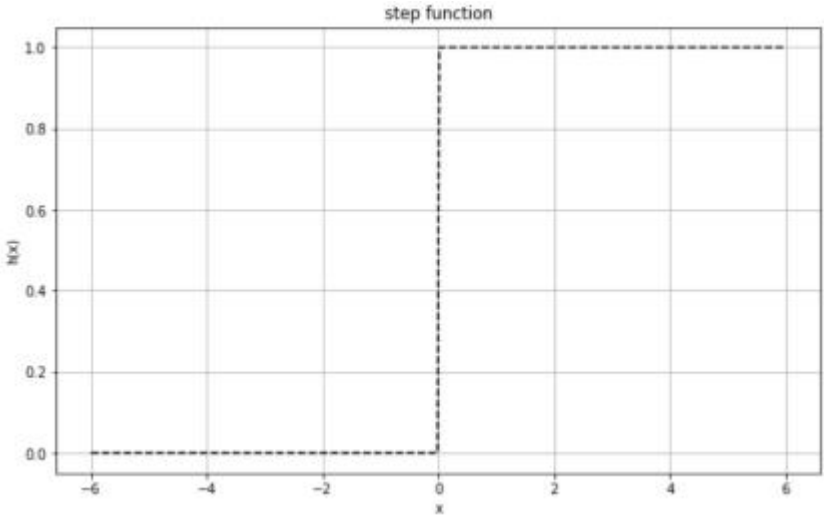
는데 이를 **활성화 함수(activation function)**라고 부른다. '활성화'라는 이름이 말해주듯 활성화 함수는 입력 신호의 가중합이 활성화를 일으키는지 정하는 역할을 한다.

자주 쓰이는 활성화 함수의 종류에 대해서 알아보자.

2-1. 계단함수 (step function)

신호의 가중합 WX^T 가 어떤 임계치 θ보다 작으면 뉴런은 출력신호를 내보내지 않고, θ보다 클 때만 출력값 1을 다른 뉴런으로 전달한다. $b = -\theta$라 하면 $z = WX^T + b$가 0보다 클 때만 출력값 1을 전달한다. 즉, 편이 b는 해당 뉴런이 얼마나 잘 활성화되는지를 나타내는 지표라고 볼 수도 있다. 신호의 흐름 상태를 1로, 신호의 차단 상태를 0으로 간단히 표현하기도 하면 다음과 같이 계단함수를 정의하여 활성화 함수로 사용할 수 있다.

$$h(x) = \begin{cases} 0 & \text{if } x \leq 0 \\ 1 & \text{if } x > 0 \end{cases}$$



이 함수는 $x = 0$**에서는 미분이 불가능**하지만, $x \neq 0$에서는 미분이 1이다. 즉, $x \neq 0$ 일 때 $h'(x) = 0$.

2-2. 로지스틱 함수 (logistic function)

계단함수는 $x = 0$에서 미분이 불가능한 함수라는 단점과, 신호의 활성/비활성 상태가 급격하게 변하는 점, 즉 불연속인 함수라는 단점이 있다. **시그모이드 함수(sigmoid function)**는 이러한 단점을 극복하기 위해 사용되는 함수로 sigmoid 라는 말은 S자형이라는 뜻이다. 시그모이드 함수의 예로 로지스틱 함수, 쌍곡탄젠트함수, 역탄젠트함수 등이 있으며 모두 미분이 잘 되는 함수이다. **로지스틱 함수의** 정의는 다음과 같다.

$$h(x) = \frac{1}{1 + e^{-ax}} = \frac{e^{ax}}{e^{ax} + 1}$$

로지스틱함수는 다음과 같은 성질을 갖는다.

① $0 < e^{ax} < e^{ax} + 1$ 이므로 $0 < h(x) < 1$이다,

② $\lim_{x \to -\infty} \frac{e^{ax}}{e^{ax}+1} = 0$, $\lim_{x \to \infty} \frac{e^{ax}}{e^{ax}+1} = 1$이다.

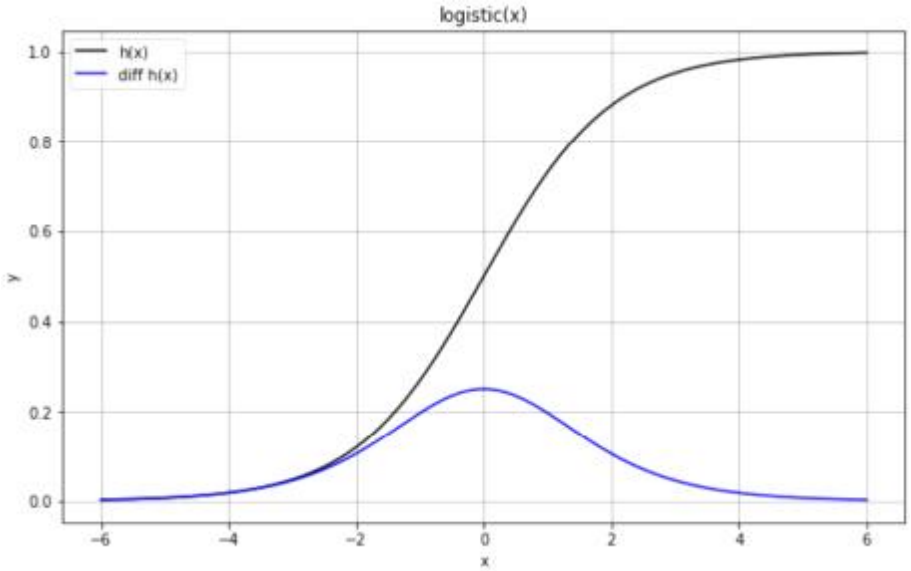
③ $h'(x) = \frac{ae^{ax}(e^{ax}+1) - e^{ax}ae^{ax}}{(e^{ax}+1)^2} = \frac{ae^{ax}}{(e^{ax}+1)^2}$

여기에서 ③번을 살펴보면, 모든 x에 대하여 $0 < h'(x) < 1$이므로 $h(x)$는 증가함수이다.

보통 활성화 함수로 $a = 1$인 경우를 많이 사용하기 때문에 $y = h(x) = \frac{e^x}{e^x+1}$을 생각해보자. 이 경우

$$y' = \frac{e^x}{(e^x+1)^2} = \frac{e^x}{e^x+1} \cdot \frac{1}{e^x+1} = y(1-y)$$

이다. 공식을 보면 x값뿐만 아니라 y**값만으로도 곡선의 변화율을 계산**할 수 있다는 뜻이다. 함수와 도함수의 그래프는 다음과 같다.



2-3. 쌍곡탄젠트 함수 ($\tanh x$, hyperbolic tangent function)

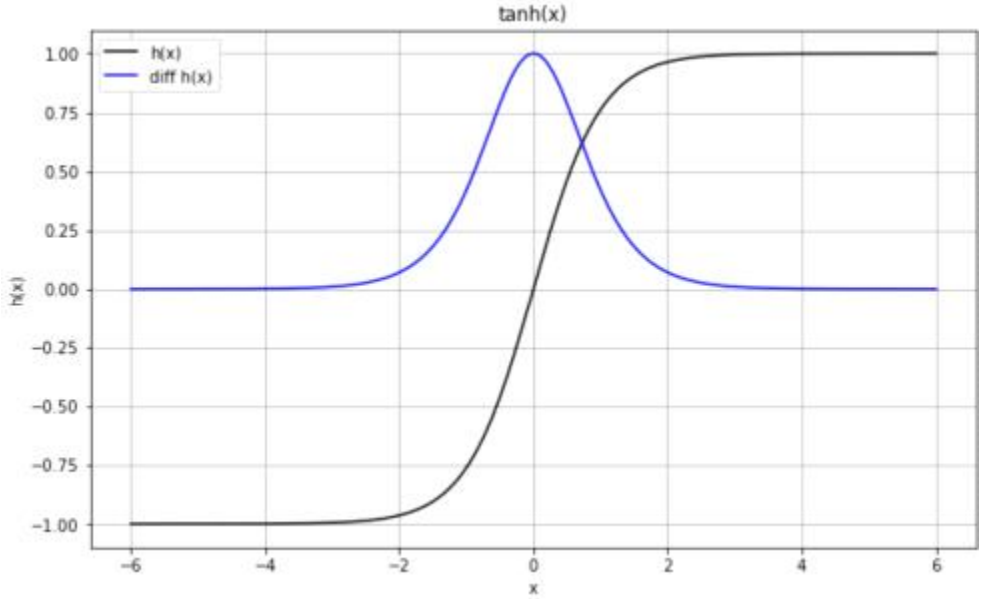
로지스틱 함수와 비슷한 함수로 쌍곡탄젠트 함수인

$$h(x) = \tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

도 많이 사용한다. tanh 함수도 로지스틱 함수와 비슷한 성질을 갖는다. 다만 $-\infty$에서의 극한이 -1이며 $y = \tanh(x)$의 미분은

$$y' = \left(\frac{2}{e^x + e^{-x}}\right)^2 = 1 - \tanh^2 x = 1 - y^2$$

이다. 함수와 도함수의 그래프는 다음과 같다.



2-4. 역탄젠트함수 ($\tan^{-1}x$, $\arctan x$)

역탄젠트함수는 탄젠트함수의 역함수로,

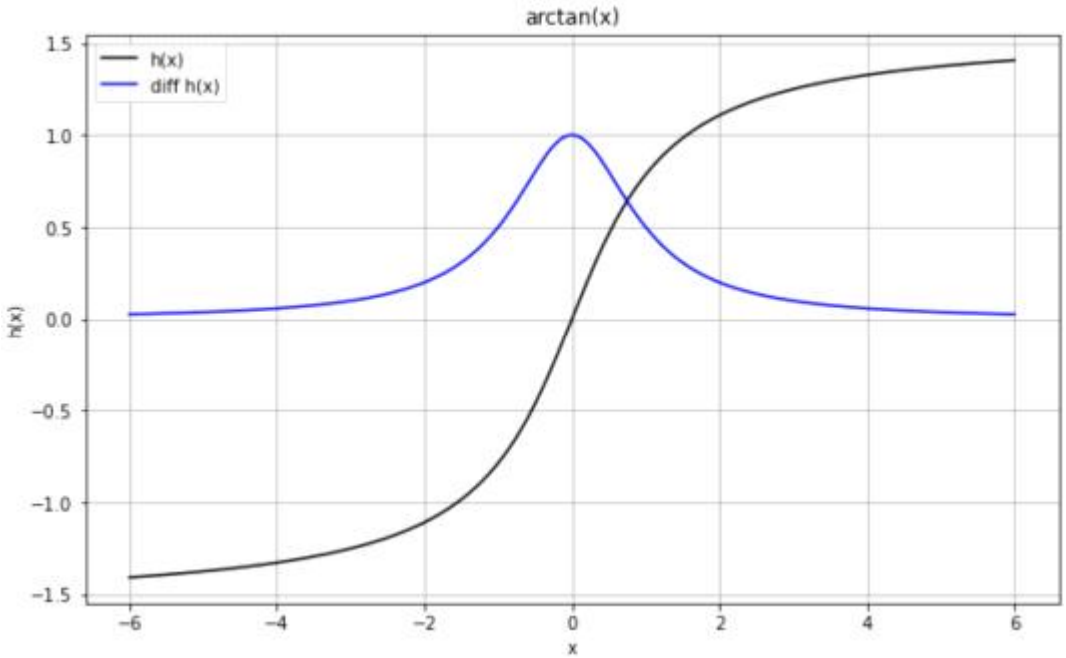
$$y = \tan^{-1}x \Leftrightarrow x = \tan y$$

로 정의된다. $\tan^{-1}x$ 대신 $\arctan x$로 표기하기도 한다.

$y = \pm\pi/2$를 점근선으로 가지며 도함수는

$$y' = \frac{1}{1+x^2} = \frac{1}{1+\tan^2 y} = \frac{1}{\sec^2 y} = \cos^2 y$$

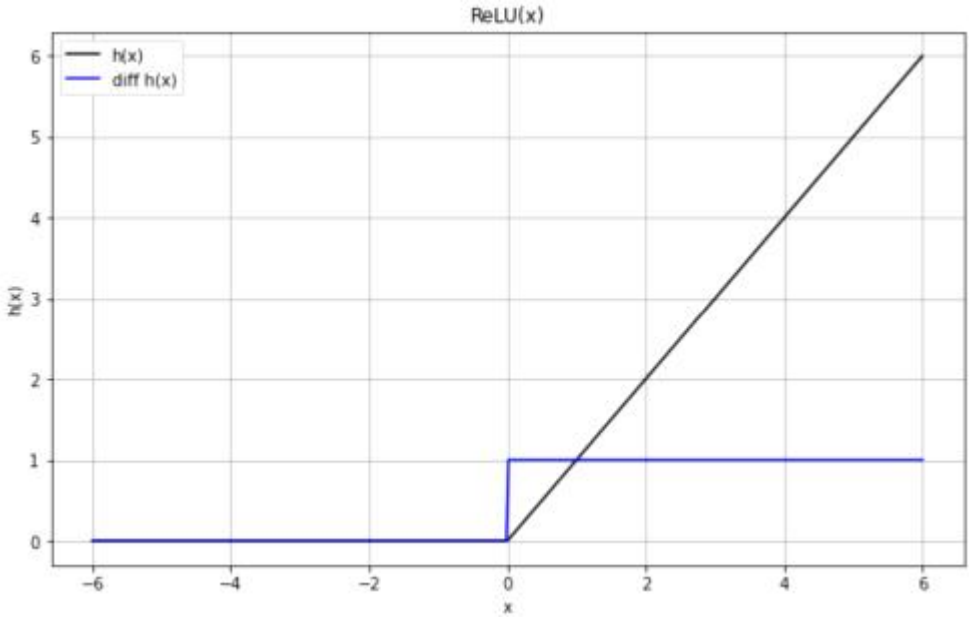
이다. 함수와 도함수의 그래프는 다음과 같다.



2-5. ReLU 함수(Rectified Linear Unit, 렐루)

계단함수와 앞의 몇 가지 시그모이드 함수 모두, 입력 신호가 아무리 커도, **출력값이 1을 넘지 않는 특징**이 있다. ReLU(Rectified Linear Unit, 렐루) 함수는 임계점이 넘었을 출력을 입력신호의 크기에 비례하게 전달한다. 렐루함수의 정의와 도함수는 다음과 같다.

$$h(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}, \quad h'(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x > 0 \end{cases}$$



위에 소개된 모든 활성화 함수의 그래프를 그리는 코드는 activation_function.ipynb 에서 확인할 수 있다.

3. 비선형 함수(nonlinear function)

이러한 활성화 함수는 모두 **비선형 함수**(nonlinear function)이다. 선형함수란 일차식으로 나타내는 함수를 말하고, 일차함수가 아닌 모든 함수를 비선형 함수라고 한다.

앞서 활성화 함수의 역할이 뉴런에서 신호의 소멸과 활성을 모델링하기 위해 만든 함수라고 소개하였지만, 그것은 어디까지나 초기 지능 연구에서 뉴런으로부터 인공신경망을 모델링하기 위한 아이디어 차원의 이야기였다. 신경망의 수리적인 구조 측면에서 설명할 때 활성화 함수는 여러 개의 선형함수와 비선형 함수의 합성으로 만들어진 임의의 함수인 것이다.

선형 함수는 아무리 여러 번 합성해도 여전히 선형 함수이다. 선형함수인 $f(x) = ax + b$와 $g(x) = cx + d$를 합성해도

$$(f \circ g)(x) = f(g(x)) = a(cx+d)+b = acx + (ad+b)$$

와 같이 일차식이 되어 여전히 선형함수이다. 선형함수를 백만 개를 더하고 빼고 합성함수로 만들어 결합해도 여전히 선형함수이다.

반면 비선형 함수인 $h(x) = \dfrac{1}{1+e^{-x}}$를 f, g와 합성하고 이를 가중치 w_1과 w_2를 사용하여 선형 결합하면

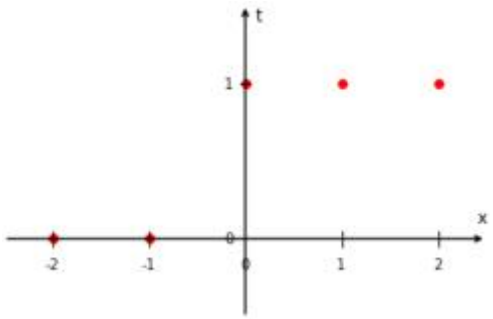
$$y = w_1(h \circ f)(x) + w_2(h \circ g)(x) + b' = \frac{w_1}{1+e^{-(ax+b)}} + \frac{w_2}{1+e^{-(cx+d)}} + b'$$

와 같은 더 복잡한 함수를 만들 수 있다. 이런 **결합을 무수히 많이 활용하면 임의의 모든 종류의 연속함수를 충분한 정확도 표현**할 수 있다. 즉, 선형 함수와 비선형 함수의 결합으로 다양한 함수를 표현하기 위해 활성화 함수를 사용한다.

4. 비선형 회귀와 분류 문제

다음과 같은 데이터가 주어져 있자고 하자.

i	x	t
0	-2	0
1	-1	0
2	0	1
3	1	1
4	2	1



i는 데이터 번호를 나타내며 $i = 0, 1, \cdots, 4$에 대하여 데이터는 (x_i, t_i) 순서쌍으로 주어졌다. x는 입력데이터이고 그에 대응하는 출력 데이터는 target을 뜻하는 t로 표시하였다. t의 값이 0또는 1로 이산적인 두 값만을 취하는데, 이렇게 **출력변수가 유한개의 값을 갖는 문제를 분류(classification) 문제**라고 한다. 특히 이 경우

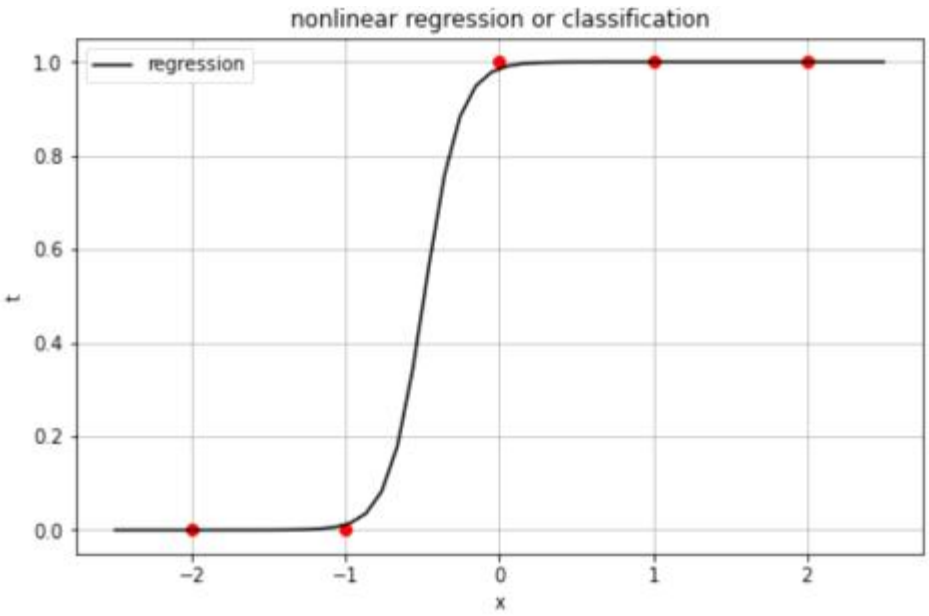
는 타겟이 두 개이므로 이 문제는 **이진 분류 (binary classification)** 라고 할 수 있다. 타겟이 세 개 이상인 문제는 **다중 분류**라고 부른다. 또한, 입력 변수 x가 1차원 데이터이므로 위 문제는 1차원 이진 분류 문제로 볼 수 있다. 입력 데이터 x가 사진 파일이나, 텍스트와 같이 고차원의 데이터인 경우는 고차원 분류 문제라고 부를 수 있다.

예를 들어, 컴퓨터가 사진을 보고 개인지 고양이인지 분류하는 문제는 사진 데이터 자체가 하나의 숫자가 아닌 고차원 데이터이고 타겟이 두 개(개 or 고양이)이므로 이진 분류 문제이다. 신경망의 입력은 사진 데이터가 될 것이고 출력은 사진이 개 사진이면 0을, 고양이 사진이면 1을 출력하도록 훈련 시키면 될 것이다.

앞서 teachable machine으로 여러 종류의 꽃 사진으로 인공신경망을 훈련시켜 꽃의 종류를 분류하는 모델을 만드는 것을 실습했었는데 이는 고차원 다중 분류 문제로 볼 수 있다. 영화 리뷰 분류기 실습 문제는, 입력 데이터가 텍스트이다. 텍스트는 단순히 하나의 숫자로 표현되지 않고 벡터로 표시되는 고차원 데이터이고, 출력이 긍정(1) 또는 부정(0)으로 두 개이므로 이는 고차원 이진 분류 문제로 볼 수 있다.

이제 비선형 함수인 활성화 함수의 활용에 대해서 알아보자.

1. 예측 모델 만들기



위 그림은 로지스틱 함수 $h(x) = 1/(1+e^{-x})$와 선형결합 $y = ax + b$을 이용하여

$$z = f(x) = h(y(x)) = \frac{1}{1+e^{-(ax+b)}}$$

형태의 함수로 비선형적으로 주어진 데이터 포인트 (x_i, t_i)를 예측하는 함수를 만든 것이다. f의 출력은 연속적인 값이므로 이를 이산적인 값으로 나타내려면 추가적으로 0또는 1로 분류하는 작업이 필요한데, 만약 $f(x_i)$의 값이 어떤 임계점, 예를 들어 0.5를 넘으면 대응되는 타겟값을 $t(x_i)=1$로 정하고, 0.5를 넘지 않으면 $t(x_i)=0$으로 정하면 된다. 즉, 분류 문제는 회귀와 마찬가지로 주어진 데이터를 최적으로 근사하는 비선형 (연속) 함수를 찾고, 입력에 대해 **예측된 함숫값을 이산적인 값으로 바꾸어 주는 작업**을 추가적으로 수행하면 된다.

2. 예측 모델의 모수(parameter) 찾기

이제 예측 함수 f를 구성하는 모수 a와 b를 데이터로부터 찾아보자.

- **손실 함수 정의하기**

선형회귀에서와 마찬가지로 손실함수를

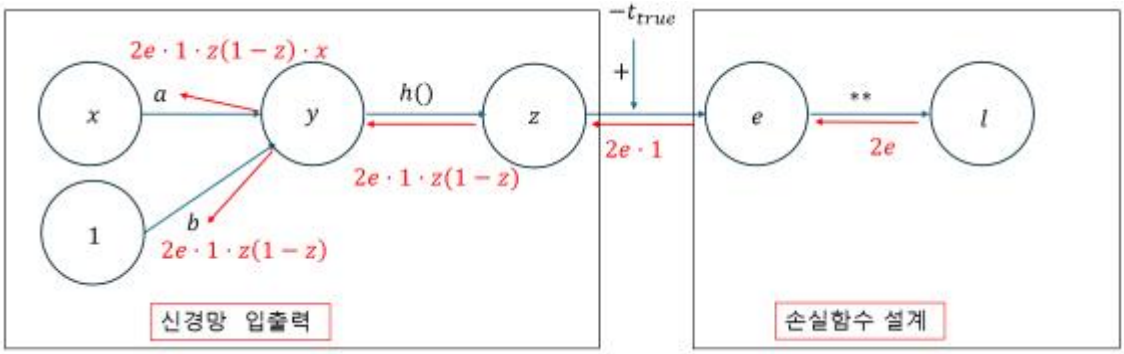
$$L(a,b) = \frac{1}{N}\sum_{i=0}^{N}(f(x_i)-t_i)^2 = \frac{1}{N}\sum_{i=0}^{N}(h(ax_i+b)-t_i)^2$$

로 정의하면 $L(a,b)$는 a와 b에 대한 이변수함수이다. 합성함수 미분법으로 $\frac{\partial L}{\partial a}$, $\frac{\partial L}{\partial b}$를 계산하면

$$\begin{cases} \frac{\partial L}{\partial a} = \frac{2}{N}\sum_{i=0}^{N}(h(y_i)-t_i)\frac{dh}{dy_i}\frac{\partial y_i}{\partial a} = \frac{2}{N}\sum_{i=0}^{N}(h(y_i)-t_i)z_i(1-z_i)x_i \\ \frac{\partial L}{\partial b} = \frac{2}{N}\sum_{i=0}^{N}(h(y_i)-t_i)\frac{dh}{dy_i}\frac{\partial y_i}{\partial b} = \frac{2}{N}\sum_{i=0}^{N}(h(y_i)-t_i)z_i(1-z_i) \end{cases} \quad \cdots (1)$$

이다. (여기서 $h'(y) = z(1-z)$라는 사실을 이용하였다.)

이 과정을 변수 치환과 함께 더 세밀하게 분석해 보자.



위의 신경망 그래프와 같이 입력 x에서 출력 z, 그리고 손실 l 까지의 계산을 구체

적으로 따져보며 순방향(왼쪽에서 오른쪽)으로의 흐름, **순전파**를 계산하고, l에서 매개변수 a와 b까지의 역방향(오른쪽에서 왼쪽)으로의 변화율의 전파, **역전파**를 생각 해보자.

- **순전파**

우선 각 단계에 등장하는 변수를 치환하여 정의하자. 입력값 x와 a,b의 일차결합을 $y = ax + b$, 활성화 함수를 통과한 값을 $z = h(y)$, 타겟과의 차이를 나타내는 오차를 $e = z - t$, 제곱손실을 $l = e^2$이라 하면, 모든 데이터 (x_i, t_i)에 대한 손실함수 L은

$$L(a,b) = \frac{1}{N}\sum_{i=0}^{N} l_i, \qquad \begin{cases} y = ax + b \\ z = h(y) \\ e = z - t \\ l = e^2 \end{cases} \quad \cdots (2)$$

이다. 데이터가 입력되면 신경망 네트워크는 (2)에 주어진 식들을 위에서 아래로 차례로 계산해나간다. 이것이 순방향 전파식이다. (식 (2)에서 때에 따라 데이터 번호를 나타내는 아래첨자 i는 생략한다.)

- **역전파**

이제 변화율의 역방향으로의 전파를 계산해보자. 식 (2)에서 각 변수들 사이의 변화율을 구해보면

$$\frac{dl}{de} = 2e, \quad \frac{de}{dz} = 1, \quad \frac{dz}{dy} = z(1-z), \quad \frac{\partial y}{\partial a} = x, \quad \frac{\partial y}{\partial b} = 1$$

이다. 가중치 a와 b에 의존하는 모든 변수들의 상태를 추적하면서 합성함수 미분법을 적용하면

$$\begin{cases} \dfrac{\partial l}{\partial a} = \dfrac{dl}{de}\dfrac{de}{dz}\dfrac{dz}{dy}\dfrac{\partial y}{\partial a} = 2e \cdot 1 \cdot z(1-z) \cdot x \\ \dfrac{\partial l}{\partial b} = \dfrac{dl}{de}\dfrac{de}{dz}\dfrac{dz}{dy}\dfrac{\partial y}{\partial b} = 2e \cdot 1 \cdot z(1-z) \end{cases} \quad \cdots (3)$$

이므로

$$\begin{cases} \dfrac{\partial L}{\partial a} = \dfrac{2}{N}\displaystyle\sum_{i=0}^{N} e_i z_i (1-z_i) x_i \\ \dfrac{\partial L}{\partial b} = \dfrac{2}{N}\displaystyle\sum_{i=0}^{N} e_i z_i (1-z_i) \end{cases}$$

이며 식(1)과 동일한 결과를 얻었다.

- **역전파로 얻은 그래디언트를 활용하여 최적의 모수 찾기**

이제 적당한 초깃값 (a_0, b_0)와 학습률 α에 대해서 경사하강법

$$\begin{cases} a_{n+1} = a_n - \alpha \times \dfrac{\partial L}{\partial a}(a_n, b_n) \\ b_{n+1} = b_n - \alpha \times \dfrac{\partial L}{\partial b}(a_n, b_n), n = 0, 1, \cdots, epochs \end{cases}$$

을 반복적으로 적용하여 주어진 데이터를 가장 잘 나타낼 수 있는 모델의 모수를 찾아나간다.

여기에서 한 가지 중요한 것은 순방향 전파시에 계산되었던 변수 y, z, e, l의 값**들을 저장하였다가 역방향으로의 변화율 전파 계산에 활용**할 수 있다는 것이다.

예를 들어, 초깃값 $(a, b) = (1, 2)$와 데이터 $(x, t) = (1, 1)$에 대해서 순방향 전파를 해보면

$$y = 3,\ z = 1/(1+e^{-3}),\ e = 1/(1+e^{-3}) - 1,\ l = (1/(1+e^{-3}) - 1)^2$$

이고, 역방향 전파에서

$$\frac{dl}{de} = 2/(1+e^{-3}) - 2,\ \frac{de}{dz} = 1,$$

$$\frac{dz}{dy} = z(1-z) = \frac{e^{-3}}{(1+e^{-3})^2},\ \frac{\partial y}{\partial a} = 1,\ \frac{\partial y}{\partial b} = 1$$

로 계산할 수 있다. 따라서 식 (3)에 따라 $\dfrac{\partial l}{\partial a}, \dfrac{\partial l}{\partial b}$를 순차적으로 곱해서 계산할 수 있다.

경사하강법을 적용하는 코드는 아래와 같으며 학습 과정까지 보여주는 자세한 코드는 1_layers_classification.ipynb에서 확인할 수 있다.

```python
import numpy as np
import matplotlib.pyplot as plt

# 1. 데이터 설정
N = 5
x = np.array([-2,-1,0,1,2])
t = np.array([0,0,1,1,1]) #targets

phi = lambda u: 1/(1+np.exp(-u))
dphi = lambda u:phi(u)*(1-phi(u))

# 2. 초기 파라미터 설정
w = 0.0  # 기울기
b = 0.0  # 절편

# 3. 하이퍼파라미터
learning_rate = 0.1
epochs = 2000
n = len(x)  # 데이터 개수

```

```
# 4. 경사하강법 학습 루프
cost_history = []

for i in range(epochs):
    u = w * x + b
    t_pred = phi(u)
    error = t_pred - t
    dphi_u = dphi(u)
    # 기울기 계산
    dw = (2 / n) * np.sum(error*dphi_u*x)
    db = (2 / n) * np.sum(error*dphi_u)

    # 파라미터 업데이트
    w -=  learning_rate * dw
    b -=  learning_rate * db

```